From RNN to LSTM to Transformer to GPT

A Student-Friendly Mathematical Intuition + PyTorch Documentation

Prepared for: Naim

Educational level assumed: Basic Machine Learning, ANN, loss functions, optimizers

Primary framework: PyTorch

Goal: Understand language models from the first recurrent idea all the way to modern GPT-style decoder-only Transformers.

Authorship, acknowledgment, and scope

This learning document was prepared with assistance from **ChatGPT (GPT-5.6 Luna)** as an educational guide.

Important note about authentication: this is not an official OpenAI publication, certificate, or authenticated academic textbook. The "authentication" here means authorship/provenance of this generated learning material, not an academic credential.

Acknowledgment: The mathematical explanations, intuition, pseudocode, and PyTorch examples are organized to help a student who already understands basic neural networks but finds dense textbook notation difficult.

The central teaching rule of this document is:

Understand the story first, then the equation, then the code.*

You will therefore see formulas written in simple words and small equations rather than jumping directly into dense notation.

---

0. The big picture

The evolution we will study is:

Neural Network

→ **Word Embedding**

→ **RNN**

→ **LSTM**

→ **Attention**

→ **Transformer**

→ **GPT**

→ **Modern GPT-style language model**

The important idea is that these are not completely unrelated inventions.

Each stage solves a weakness of the previous stage.

The progression

Model/idea	Main problem it solves
ANN	Learns a mapping from input to output
Word embedding	Turns discrete words/tokens into useful vectors
RNN	Gives a neural network memory across a sequence
LSTM	Makes long-term memory easier to preserve
Attention	Lets the model directly look at relevant earlier information
Transformer	Replaces recurrence with attention-based sequence processing
GPT	Uses a decoder-only Transformer to predict the next token
Modern GPT	Adds scale, better tokenization, normalization, optimization, data, inference systems, etc.

The single most important language-model idea is:

Given what I have seen so far, what token should come next?

For a sentence:

`The cat is sleeping`

a language model learns:

`The` → predict `cat`

`The cat` → predict `is`

`The cat is` → predict `sleeping`

`The cat is sleeping` → predict what comes next.

---

1. Prerequisites

You already know:

• basic neural networks
• loss functions
• optimizers
• basic machine learning
• basic ANN concepts

You therefore do **not** need to restart from linear regression or a basic perceptron.

You should be comfortable with:

• vectors
• matrices
• matrix multiplication
• derivatives at a basic level
• softmax
• cross-entropy loss

• PyTorch tensors

• forward pass and backpropagation

If matrix multiplication is still uncomfortable, spend time on it before going deep into attention.

---

2. The language-model problem

Suppose our training text is:

`I like machine learning`

A computer does not naturally understand the word `"machine"`.

We first convert text into tokens.

For a simple tokenizer:

`I` $\rightarrow$ 0

`like` $\rightarrow$ 1

`machine` $\rightarrow$ 2

`learning` $\rightarrow$ 3

So:

`I like machine learning`

becomes:

`[0, 1, 2, 3]`

But integers alone are not useful semantic representations.

The model should learn that:

• `cat` and `dog` are both animals

• `king` and `queen` have related meanings

• `Paris` and `London` are cities

• `run` and `running` are related

This leads us to embeddings.

---

3. Word embeddings

3.1 The problem with one-hot vectors

Suppose the vocabulary is:

`["cat", "dog", "car", "banana"]`

A one-hot representation might be:

`cat` = `[1, 0, 0, 0]`

`dog` = `[0, 1, 0, 0]`

`car` = `[0, 0, 1, 0]`

`banana` = `[0, 0, 0, 1]`

The problem is that every word is equally unrelated.

The mathematical distance between `cat` and `dog` is not smaller than between `cat` and `banana`.

We want the model to learn a continuous representation.

3.2 Embedding intuition

Instead of representing:

`cat` = `[1, 0, 0, 0]`

we could learn:

`cat` = `[0.72, -0.11, 0.34, 0.80]`

and:

`dog` = `[0.69, -0.09, 0.31, 0.77]`

while:

`car` = `[-0.20, 0.81, -0.44, 0.12]`

The exact numbers have no human-designed meaning.

The neural network discovers useful directions during training.

3.3 Embedding matrix

Suppose:

• vocabulary size = 10,000
• embedding size = 256

Then the embedding table has:

`10,000 × 256`

numbers.

Each row is one token's vector.

If token ID is 57:

`embedding[57]`

returns its 256-dimensional vector.

PyTorch

```
import torch
import torch.nn as nn

vocab_size = 10000
embedding_dim = 256

embedding = nn.Embedding(vocab_size, embedding_dim)

token_ids = torch.tensor([57, 123, 900])

vectors = embedding(token_ids)

print(vectors.shape)
# torch.Size([3, 256])
```

3.4 What actually happens during training?

The embedding starts with random values.

During training:

1. token enters the embedding table
2. its vector is used by the network
3. prediction creates a loss
4. backpropagation calculates how the loss depends on that vector
5. optimizer updates the vector

Therefore:

An embedding is simply a trainable lookup table.

This is a very important idea.

---

4. From ANN to sequence modeling

A normal ANN often receives a fixed input:

`x → network → prediction`

But language is sequential.

Consider:

`The dog chased the cat because it was hungry.`

The meaning of `"it"` depends on earlier words.

A language model therefore needs some form of context.

A simple idea is:

Read the sentence one token at a time and keep a memory of what has been read.

That is the basic idea behind an RNN.

---

5. RNN — Recurrent Neural Network

5.1 Core intuition

Imagine reading:

`I love machine learning`

At the first word:

`I`

you create a memory.

Then you read:

`love`

and update the memory.

Then:

`machine`

updates it again.

Then:

`learning`

updates it again.

So the model has:

• current input
• previous memory
• new memory

5.2 RNN equation without scary notation

At time step `t`, the RNN receives:

• current token representation: `x`

• previous memory: `h_old`

It produces:

• new memory: `h_new`

The core operation is:

```
new_memory =
tanh(
    input_information
    +
    previous_memory_information
)
```

More explicitly:

```
new_memory =
tanh(
    W_x × current_input
    +
    W_h × old_memory
    +
    bias
)
```

Do not memorize the letters.

Remember:

The RNN mixes the current word with its previous memory.

5.3 Why tanh?

The linear calculation could create extremely large values.

`tanh` squashes values approximately into:

`-1 ... +1`

It gives the network a nonlinear transformation.

5.4 RNN diagram

```
word 1 ■■→ [RNN] ■■→ memory 1
              ■
              ↓
word 2 ■■→ [RNN] ■■→ memory 2
              ■
              ↓
word 3 ■■→ [RNN] ■■→ memory 3
              ■
              ↓
```

```
word 4 ■■→ [RNN] ■■→ memory 4
```

The memory flows from one step to the next.

5.5 A simple PyTorch RNN

```
import torch
import torch.nn as nn

class SimpleRNNLanguageModel(nn.Module):
    def __init__(self, vocab_size, embedding_dim, hidden_dim):
        super().__init__()

        self.embedding = nn.Embedding(vocab_size, embedding_dim)

        self.rnn = nn.RNN(
            input_size=embedding_dim,
            hidden_size=hidden_dim,
            batch_first=True
        )

        self.output = nn.Linear(hidden_dim, vocab_size)

    def forward(self, tokens):
        x = self.embedding(tokens)
        h, _ = self.rnn(x)
        logits = self.output(h)
        return logits
```

Example:

```
model = SimpleRNNLanguageModel(
    vocab_size=10000,
    embedding_dim=128,
    hidden_dim=256
)

tokens = torch.randint(0, 10000, (4, 20))

logits = model(tokens)

print(logits.shape)
```

Output:

```
[4, 20, 10000]
```

Meaning:

• 4 sequences

• 20 positions

• 10,000 possible next tokens

---

6. How RNN language-model training works

Suppose the sequence is:

`I love cats`

Training pairs can be:

Input:

`I`

Target:

`love`

Then:

Input:

`I love`

Target:

`cats`

For a longer sequence:

```
Input:  I   love   cats
Target: love cats   <END>
```

The model produces logits for every position.

Then cross-entropy compares the predicted distribution with the correct next token.

6.1 Cross entropy intuition

Suppose the correct next word is:

`cats`

The model predicts:

```
cats       0.70
dogs       0.15
cars       0.10
banana     0.05
```

The model gets rewarded because it assigned high probability to the correct answer.

If it predicts:

```
cats       0.01
dogs       0.60
cars       0.20
banana     0.19
```

the loss becomes large.

The language model therefore learns:

Put high probability on the correct next token.

---

7. The RNN problem: long-term memory

Consider:

`The student who studied mathematics for many hours eventually solved the difficult problem because he understood it.`

The model may need information from far earlier in the sequence.

RNNs repeatedly transform the memory:

```
memory 1
   ↓
memory 2
   ↓
memory 3
   ↓
memory 4
   ↓
...
```

During backpropagation, gradients also travel backward through many time steps.

Repeated multiplication can make gradients:

• extremely small

• extremely large

These are called:

vanishing gradients

and

exploding gradients

---

8. LSTM — Long Short-Term Memory

8.1 Why LSTM exists

LSTM was designed to give recurrent networks a better mechanism for deciding:

• what information to keep

• what information to forget

• what information to expose

Think of an LSTM as a small controller attached to the memory.

8.2 Two kinds of memory

LSTM has:

Cell state

Long-term memory.

Think:

"What important information should travel forward?"

Hidden state

Current working information.

Think:

"What information should I expose right now?"

---

9. LSTM gates — intuitive view

LSTM uses three major gates.

Forget gate

Question:

What old information should I throw away?

Input gate

Question:

What new information should I store?

Output gate

Question:

What part of the memory should I reveal as the current hidden state?

---

10. LSTM equations without notation overload

At each step, the model looks at:

• current input

• previous hidden state

It calculates three decisions.

Forget decision

```
forget =
sigmoid(
    input_effect
    +
    old_hidden_effect
)
```

Values are between:

`0 and 1`

Interpretation:

• 0 → forget completely

• 1 → keep completely

Input decision

The model creates a candidate memory:

```
candidate =
tanh(
    input_effect
    +
    old_hidden_effect
)
```

Then decides how much to write:

```
new_memory =
forget × old_memory
+
input_gate × candidate
```

This equation is worth understanding deeply.

It says:

New memory = some old memory + some new information.

Output decision

Finally:

```
new_hidden =
output_gate × tanh(new_memory)
```

Again:

Hidden state is selected information from the cell memory.

---

11. Why LSTM helps

Suppose the model has learned:

`The cat ...`

Many words later it needs to remember:

`cat`

The cell state can carry important information forward with less destructive repeated transformation than a simple RNN.

The gates can keep important information while discarding irrelevant information.

---

12. PyTorch LSTM

```
import torch
import torch.nn as nn

class LSTMLanguageModel(nn.Module):
    def __init__(self, vocab_size, embedding_dim, hidden_dim):
        super().__init__()

        self.embedding = nn.Embedding(
            vocab_size,
            embedding_dim
        )

        self.lstm = nn.LSTM(
            input_size=embedding_dim,
            hidden_size=hidden_dim,
            batch_first=True
        )

        self.output = nn.Linear(
            hidden_dim,
            vocab_size
        )

    def forward(self, tokens):
        x = self.embedding(tokens)

        h, _ = self.lstm(x)

        logits = self.output(h)

        return logits
```

---

13. RNN vs LSTM

Feature	RNN	LSTM
Basic memory	Yes	Yes
Gates	No	Yes
Long-term dependencies	Difficult	Better
Architecture	Simple	More complex
Training	Sequential	Sequential
Parallelism across sequence	Poor	Poor
Main historical importance	Introduced recurrence	Improved recurrent memory

But LSTM still has a major limitation:

It processes the sequence step by step.

That becomes a serious problem for large-scale training.

This motivates attention.

---

14. Attention — the key conceptual breakthrough

Imagine this sentence:

`The animal didn't cross the road because it was tired.`

To understand `"it"`, the model should look at relevant earlier words.

Instead of forcing one fixed memory vector to contain everything, attention says:

At this position, look directly at the other positions and decide which ones matter.

This is a huge conceptual change.

---

15. Attention with a simple example

Suppose the current word is:

`it`

Previous words:

```
The animal didn't cross the road because it
    ↑       ↑                          ↑
```

The model can assign different importance:

```
The       0.02
animal    0.75
didn't    0.03
cross     0.04
the       0.02
road      0.08
because   0.04
```

The important word is likely:

`animal`

The model uses these weights to build a context representation.

---

16. Query, Key, Value intuition

Attention uses three concepts.

Query

What am I looking for?

Key

What information does each token advertise?

Value

What information should I actually retrieve?

A useful analogy is a library.

You ask a question:

Query

Books have searchable labels:

Keys

The actual book contents are:

Values

The attention mechanism compares the query with keys and retrieves values from the most relevant ones.

---

17. The mathematical heart of attention

For one token:

```
Query = current representation × query weights
Key   = token representation × key weights
Value = token representation × value weights
```

We then compare the query with every key.

A simple similarity is:

```
similarity = Query × Key
```

Large similarity means:

"These two pieces of information are related."

---

18. Turning similarity into probabilities

The raw similarity numbers are passed through softmax.

Example:

```
raw scores:
[2.0, 1.0, 0.1]
```

Softmax converts them into positive values that add up to 1.

For example:

```
[0.66, 0.24, 0.10]
```

Now they can be interpreted as attention weights.

---

19. Creating the attention output

Suppose values are:

```
V1
V2
V3
```

and attention weights are:

```
0.66
0.24
0.10
```

Then:

```
attention_output =
0.66 × V1
+
0.24 × V2
+
0.10 × V3
```

So the model creates a weighted mixture of information.

---

20. Scaled dot-product attention

The standard Transformer equation is commonly written compactly as:

```
Attention(Q, K, V)
=
softmax(
    QK■ / √d
)
V
```

Do not memorize this blindly.

Read it as four operations:

Step 1

Compare every Query with every Key:

```
Q × K■
```

Step 2

Scale the scores:

```
divide by √d
```

This prevents the numbers from becoming too large when vector dimensions are large.

Step 3

Convert scores to attention probabilities:

```
softmax(...)
```

Step 4

Use those probabilities to mix Values:

```
probabilities × V
```

That is the whole idea.

---

21. Why divide by √d?

Suppose vectors have many dimensions.

Their dot products can become large.

Large softmax inputs can make softmax extremely sharp:

```
[0.999999, 0.000001, 0]
```

That can produce poor gradients.

Dividing by the square root of the dimension keeps the scale more manageable.

---

22. Self-attention

Self-attention means:

Every token looks at other tokens in the same sequence.

Example:

```
I love machine learning
```

The representation for:

`learning`

can attend to:

`I`

`love`

`machine`

and itself.

So every token can gather contextual information.

---

23. Causal attention

For GPT, there is a critical rule:

A token must not look into the future.

Suppose:

```
I love machine learning
```

When predicting `machine`, the model may use:

```
I
love
```

but not:

```
learning
```

This is called:

causal self-attention

or

masked self-attention.

---

24. Causal attention mask

Conceptually:

```
         I  love  machine  learning
I        ✓   X      X        X
love     ✓   ✓      X        X
machine  ✓   ✓      ✓        X
learning ✓   ✓      ✓        ✓
```

The `X` positions are blocked.

In code, these positions receive a very negative number before softmax, so their probability becomes approximately zero.

---

25. Attention in PyTorch

A simple educational implementation:

```
import math
import torch

def self_attention(x):
    d = x.size(-1)

    Q = x
    K = x
    V = x

    scores = Q @ K.transpose(-2, -1)

    scores = scores / math.sqrt(d)

    weights = torch.softmax(scores, dim=-1)

    output = weights @ V

    return output
```

This is not yet a complete Transformer attention layer because real attention learns separate Q/K/V projections and usually uses multiple heads.

But it exposes the core mathematics.

---

26. Learned Query, Key, Value projections

Instead of using the same `x` directly, we learn three transformations:

```
Q = x @ Wq
K = x @ Wk
V = x @ Wv
```

where:

• `Wq` learns how to ask questions

• `Wk` learns how to advertise information

• `Wv` learns what information to provide

In PyTorch:

```
self.q = nn.Linear(dim, dim)
self.k = nn.Linear(dim, dim)
self.v = nn.Linear(dim, dim)
```

---

27. Multi-head attention

One attention mechanism can learn one type of relationship.

But language contains many relationships.

For example:

• subject ↔ verb

• adjective ↔ noun

• pronoun ↔ noun

• nearby syntax

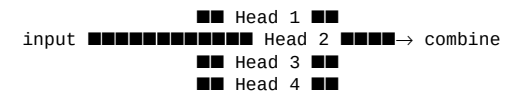
• long-distance dependency

• semantic relationship

So Transformers use multiple attention heads.

Each head gets its own learned Q/K/V projections.

Conceptually:



---

28. Multi-head attention mathematically

Each head performs:

```
head_i =
Attention(
    Q_i,
    K_i,
    V_i
)
```

Then:

```
all_heads =
concatenate(
    head_1,
    head_2,
    ...,
    head_n
)
```

Finally:

```
output =
all_heads × output_weights
```

The important intuition:

Different heads can learn different relationships.

---

29. Transformer

The Transformer architecture was introduced to make sequence modeling heavily based on attention rather than recurrence.

A simplified Transformer block is:

```
Input
  ↓
Self-Attention
  ↓
Add & Normalize
  ↓
Feed-Forward Network
  ↓
Add & Normalize
  ↓
Output
```

Modern GPT implementations use variants of this basic idea.

---

30. Why Transformers changed everything

RNN:

```
token 1
  ↓
token 2
  ↓
token 3
  ↓
token 4
```

The sequence is inherently sequential.

Transformer:

```
token 1 ■■
token 2 ■■■→ attention
token 3 ■■
token 4 ■■
```

During training, many positions can be processed in parallel.

This is one of the biggest reasons Transformers scale so effectively on GPUs.

---

31. But how does a Transformer know token order?

Attention itself does not automatically know whether:

```
dog bites man
```

is different from:

```
man bites dog
```

The model therefore needs positional information.

---

32. Positional information

A simple idea is:

```
token representation
+
position representation
```

So the model receives both:

• what token this is

• where it occurs

There are several approaches.

Historical Transformers used sinusoidal positional encodings.

Modern language models often use learned or relative-style positional methods such as rotary positional embeddings.

---

33. Rotary Position Embedding — intuition

RoPE is widely used in modern decoder-only language models.

Instead of simply adding a position vector to each token, RoPE rotates parts of the Query and Key vectors according to position.

The important effect is:

The attention score can encode relative position information.

You do not need to derive the trigonometry before understanding GPT.

Learn the conceptual order:

```
token embedding
→ Q/K projections
→ position-dependent rotation
→ attention
```

---

34. Transformer feed-forward network

After attention, every token representation goes through a small neural network.

Conceptually:

```
x
↓
Linear
↓
nonlinearity
↓
Linear
↓
output
```

For example:

```
self.ffn = nn.Sequential(
    nn.Linear(dim, 4 * dim),
    nn.GELU(),
    nn.Linear(4 * dim, dim)
)
```

The attention layer answers:

Which information should I look at?

The feed-forward layer answers:

Given the information I now have, how should I transform it?

---

35. Residual connections

A Transformer block does not simply replace the old representation.

It adds the new information to the old representation.

Conceptually:

```
new = old + transformation(old)
```

This is called a residual connection.

Why?

It helps information and gradients travel through deep networks.

---

36. Layer normalization

Deep Transformers need stable activations.

LayerNorm normalizes information across the hidden dimensions for each token representation.

The exact implementation details vary, but intuition is:

Keep the internal representation at a manageable scale so deep networks are easier to train.

---

37. A Transformer block in PyTorch

Educational version:

```
import torch
import torch.nn as nn

class TransformerBlock(nn.Module):
    def __init__(self, dim, heads):
        super().__init__()

        self.norm1 = nn.LayerNorm(dim)

        self.attention = nn.MultiheadAttention(
            embed_dim=dim,
            num_heads=heads,
            batch_first=True
        )

        self.norm2 = nn.LayerNorm(dim)

        self.ffn = nn.Sequential(
            nn.Linear(dim, 4 * dim),
            nn.GELU(),
            nn.Linear(4 * dim, dim)
        )

    def forward(self, x):
        normalized = self.norm1(x)

        attention_output, _ = self.attention(
            normalized,
            normalized,
            normalized
        )

        x = x + attention_output

        x = x + self.ffn(self.norm2(x))

        return x
```

This is intentionally simple.

Production GPT implementations add causal masking, positional methods, optimized attention kernels, dropout choices, weight tying choices, and other engineering details.

---

38. GPT

GPT stands for:

Generative Pre-trained Transformer

The most important architectural idea:

GPT is a decoder-only Transformer trained to predict the next token.

For example:

```
Input:
The cat is

Target:
sleeping
```

The model produces a probability distribution over the vocabulary.

---

39. GPT training objective

Suppose the training sequence is:

```
The cat is sleeping
```

Inputs:

```
The
The cat
The cat is
```

Targets:

```
cat
is
sleeping
```

The model tries to maximize the probability of the correct next token.

A standard loss can be written simply as:

```
Loss = average negative log probability of the correct next token
```

The mathematical form is:

```
L = - average[ log P(correct next token) ]
```

This is much easier to remember than a large indexed equation.

---

40. Why negative log?

Suppose correct-token probability is:

`0.9`

Then:

`-log(0.9)` is small.

Good prediction → small loss.

If probability is:

`0.01`

then:

`-log(0.01)` is large.

Bad prediction → large loss.

Therefore the loss strongly punishes the model for assigning low probability to the correct token.

---

41. Teacher forcing

During training, GPT receives the real previous tokens.

Example:

```
Input:
The cat is

Target:
cat is sleeping
```

The model does not have to generate the first word before learning the second.

This makes training efficient.

At inference time, however, the model generates one token at a time.

---

42. GPT architecture

A simplified GPT looks like:

```
tokens
  ↓
token embeddings
  +
positional information
  ↓
Transformer block 1
  ↓
```

```
Transformer block 2
  ↓
...
  ↓
Transformer block N
  ↓
LayerNorm
  ↓
Linear vocabulary projection
  ↓
logits
  ↓
softmax
  ↓
next-token probabilities
```

---

43. Token embedding + positional representation

Suppose:

• vocabulary = 50,000

• model dimension = 768

Token embedding table:

```
50,000 × 768
```

For a sequence of 128 tokens:

```
128 × 768
```

is passed through the Transformer.

The Transformer changes each token's representation into a context-aware representation.

---

44. Logits

At the final layer, each token representation is mapped to vocabulary-sized scores.

If vocabulary size is:

`50,000`

then each position produces:

`50,000 logits`

Example:

```
token A → 1.2
token B → -0.4
token C → 5.7
...
```

These are not probabilities yet.

Softmax converts them into probabilities.

---

45. Why logits are used with CrossEntropyLoss

In PyTorch:

```
loss_fn = nn.CrossEntropyLoss()
loss = loss_fn(logits, targets)
```

You normally pass raw logits.

You do not need to call softmax manually before `CrossEntropyLoss`.

PyTorch's cross-entropy implementation internally performs the appropriate log-softmax + negative log-likelihood calculation.

---

46. Complete small GPT in PyTorch

The following is an educational GPT-style model.

```
import math
import torch
import torch.nn as nn

class CausalSelfAttention(nn.Module):
    def __init__(self, dim, heads, block_size):
        super().__init__()

        assert dim % heads == 0

        self.heads = heads
        self.head_dim = dim // heads

        self.qkv = nn.Linear(dim, 3 * dim)
        self.out = nn.Linear(dim, dim)

        mask = torch.tril(
            torch.ones(block_size, block_size)
        )

        self.register_buffer(
            "mask",
            mask.view(1, 1, block_size, block_size)
        )

    def forward(self, x):
        B, T, C = x.shape

        qkv = self.qkv(x)

        q, k, v = qkv.chunk(3, dim=-1)

        q = q.view(
            B, T, self.heads, self.head_dim
        ).transpose(1, 2)

        k = k.view(
            B, T, self.heads, self.head_dim
        ).transpose(1, 2)

        v = v.view(
            B, T, self.heads, self.head_dim
        ).transpose(1, 2)

        scores = q @ k.transpose(-2, -1)

        scores = scores / math.sqrt(self.head_dim)

        scores = scores.masked_fill(
            self.mask[:, :, :T, :T] == 0,
            float("-inf")
        )

        weights = torch.softmax(
            scores,
```

```
            dim=-1
        )

        y = weights @ v

        y = y.transpose(1, 2).contiguous()

        y = y.view(B, T, C)

        return self.out(y)


class MLP(nn.Module):
    def __init__(self, dim):
        super().__init__()

        self.net = nn.Sequential(
            nn.Linear(dim, 4 * dim),
            nn.GELU(),
            nn.Linear(4 * dim, dim)
        )

    def forward(self, x):
        return self.net(x)


class GPTBlock(nn.Module):
    def __init__(self, dim, heads, block_size):
        super().__init__()

        self.norm1 = nn.LayerNorm(dim)
        self.attention = CausalSelfAttention(
            dim,
            heads,
            block_size
        )

        self.norm2 = nn.LayerNorm(dim)
        self.mlp = MLP(dim)

    def forward(self, x):
        x = x + self.attention(
            self.norm1(x)
        )

        x = x + self.mlp(
            self.norm2(x)
        )

        return x


class TinyGPT(nn.Module):
    def __init__(
        self,
        vocab_size,
        block_size,
        dim=256,
        heads=8,
        layers=6
    ):
        super().__init__()

        self.token_embedding = nn.Embedding(
            vocab_size,
            dim
        )

        self.position_embedding = nn.Embedding(
            block_size,
            dim
        )

        self.blocks = nn.ModuleList([
            GPTBlock(
                dim,
                heads,
                block_size
            )
            for _ in range(layers)
        ])

        self.norm = nn.LayerNorm(dim)

        self.lm_head = nn.Linear(
            dim,
            vocab_size,
            bias=False
        )

    def forward(self, tokens, targets=None):
```

```
        B, T = tokens.shape

        positions = torch.arange(
            T,
            device=tokens.device
        )

        x = self.token_embedding(tokens)
        x = x + self.position_embedding(positions)

        for block in self.blocks:
            x = block(x)

        x = self.norm(x)

        logits = self.lm_head(x)

        loss = None

        if targets is not None:
            B, T, V = logits.shape

            loss = nn.functional.cross_entropy(
                logits.view(B * T, V),
                targets.view(B * T)
            )

        return logits, loss
```

---

47. Understanding the TinyGPT code

Do not try to understand all of it at once.

Follow this path.

Step 1

```
tokens
```

are integer IDs.

Example:

```
[10, 25, 91, 17]
```

Step 2

```
self.token_embedding(tokens)
```

turns IDs into vectors.

Now:

```
4 tokens
```

becomes:

```
4 vectors
```

Step 3

Position information is added.

Now each vector knows approximately:

"What token am I and where am I?"

Step 4

The Transformer blocks repeatedly update the vectors.

Step 5

Causal attention allows each position to use only current and previous positions.

Step 6

The final linear layer converts each hidden vector into vocabulary scores.

Step 7

Cross-entropy compares the scores against the correct next tokens.

---

48. The tensor-shape journey

This is one of the most useful things to understand.

Suppose:

```
batch = 4
sequence length = 32
model dimension = 256
vocabulary = 10,000
```

Input:

```
[4, 32]
```

Embedding:

```
[4, 32, 256]
```

Attention:

```
[4, 32, 256]
```

Transformer blocks:

```
[4, 32, 256]
```

Final vocabulary projection:

```
[4, 32, 10000]
```

Loss flattens the first two dimensions:

```
[128, 10000]
```

against:

```
[128]
```

where:

`128 = 4 × 32`

---

49. Multi-head tensor shapes

Suppose:

```
B = 4
T = 32
C = 256
heads = 8
```

Then:

```
head_dim = 256 / 8 = 32
```

After splitting:

```
Q shape = [4, 8, 32, 32]
K shape = [4, 8, 32, 32]
V shape = [4, 8, 32, 32]
```

Attention scores:

```
Q × K■
```

produce:

```
[4, 8, 32, 32]
```

The last two dimensions mean:

```
query position × key position
```

This is the attention map.

---

50. What does the attention matrix mean?

For a sequence of 4 tokens:

```
          token1 token2 token3 token4
token1     1      0      0      0
token2    0.4    0.6     0      0
token3    0.1    0.3    0.6     0
token4    0.1    0.1    0.2    0.6
```

Each row answers:

When processing this token, how much should I use from every allowed previous token?

This is one of the most important visual intuitions for Transformers.

---

51. Training a GPT

A basic training loop:

```
import torch

device = "cuda" if torch.cuda.is_available() else "cpu"

model = TinyGPT(
    vocab_size=10000,
    block_size=128,
    dim=256,
    heads=8,
    layers=6
).to(device)

optimizer = torch.optim.AdamW(
    model.parameters(),
    lr=3e-4
)

for step in range(10000):
    x, y = get_batch()

    x = x.to(device)
    y = y.to(device)

    logits, loss = model(x, y)

    optimizer.zero_grad()

    loss.backward()

    torch.nn.utils.clip_grad_norm_(
        model.parameters(),
        1.0
    )

    optimizer.step()

    if step % 100 == 0:
        print(
            step,
            loss.item()
        )
```

---

52. Why AdamW?

You already know what an optimizer does.

For GPT training, AdamW is a common choice.

You do not need to memorize its internal moving-average equations right now.

For this learning stage, remember:

AdamW updates the model parameters using gradients while handling weight decay in a way suitable for modern neural-network training.

Focus your mathematical effort on:

1. embeddings
2. attention
3. causal masking
4. Transformer block
5. cross-entropy
6. next-token prediction

---

53. Getting training batches

Suppose tokenized data is:

```
[10, 20, 31, 42, 51, 67, 72]
```

With block size 4:

Input:

```
10 20 31 42
```

Target:

```
20 31 42 51
```

Another sample:

Input:

```
20 31 42 51
```

Target:

```
31 42 51 67
```

This one-token shift is fundamental to GPT training.

---

54. Batch creation code

```
def get_batch(data, batch_size, block_size, device):
    starts = torch.randint(
        0,
        len(data) - block_size - 1,
        (batch_size,)
    )

    x = torch.stack([
        data[i:i + block_size]
        for i in starts
    ])

    y = torch.stack([
        data[i + 1:i + block_size + 1]
        for i in starts
    ])

    return x.to(device), y.to(device)
```

---

55. Generation

Training asks:

What is the correct next token?

Generation repeatedly asks:

What token should I produce next?

Example:

```
Prompt:
The cat

Model predicts:
is

Now input becomes:
The cat is

Model predicts:
sleeping

Now input becomes:
The cat is sleeping
```

This repeats until a stopping condition is reached.

---

56. Basic generation code

```
@torch.no_grad()
def generate(model, tokens, max_new_tokens):
    model.eval()

    for _ in range(max_new_tokens):

        context = tokens[:, -model.position_embedding.num_embeddings:]

        logits, _ = model(context)

        logits = logits[:, -1, :]

        probabilities = torch.softmax(
            logits,
            dim=-1
        )

        next_token = torch.multinomial(
            probabilities,
            num_samples=1
        )

        tokens = torch.cat(
            [tokens, next_token],
            dim=1
        )

    return tokens
```

---

57. Greedy decoding vs sampling

Suppose the model predicts:

```
cat      0.50
```

```
dog      0.30
bird     0.15
car      0.05
```

Greedy

Always choose:

`cat`

This is deterministic.

Sampling

Randomly choose according to the probability distribution.

This can create more varied text.

---

58. Temperature

Temperature changes how sharp the probability distribution is.

Conceptually:

Lower temperature

More conservative.

The highest-probability choices dominate.

Higher temperature

More randomness.

Less-probable choices become more likely.

A simplified formula is:

```
new_logits = logits / temperature
```

Then softmax is applied.

Remember:

Temperature changes randomness; it does not give the model new knowledge.

---

59. Top-k sampling

Instead of sampling from the entire vocabulary:

1. keep only the top `k` candidates

2. remove everything else

3. sample from those candidates

Example:

```
k = 5
```

Only the five highest-scoring tokens remain.

---

60. Top-p sampling

Top-p keeps the smallest group of tokens whose combined probability reaches a threshold.

Example:

```
p = 0.9
```

The model keeps enough high-probability candidates to cover roughly 90% of the probability mass.

This is called nucleus sampling.

---

61. Tokenization: why modern GPT does not usually use words

A word-level tokenizer has a problem.

Suppose a word never appeared in training:

`unbelievability`

A character tokenizer can represent it:

```
u n b e l i e v a b i l i t y
```

but sequences become long.

Modern language models therefore commonly use subword-style tokenization.

Example:

```
unbelievable
```

might be represented approximately as:

```
un + believable
```

The exact tokenization depends on the tokenizer.

---

62. BPE intuition

BPE means:

Byte Pair Encoding

Start with smaller units and repeatedly merge frequent neighboring units.

For example, suppose the corpus frequently contains:

```
l o
```

Merge:

```
lo
```

Then:

```
lo w
```

might become:

```
low
```

The actual algorithms used by modern tokenizers vary, but the main idea is:

Frequently occurring pieces become reusable tokens.

---

63. Why tokenization matters

Tokenization determines:

• sequence length

• vocabulary size

• memory usage

• training efficiency

• how text is represented

A good tokenizer balances:

```
vocabulary size
vs
sequence length
```

Too many tiny tokens:

• longer sequences

• more computation

Too many giant tokens:

• huge vocabulary

• poor handling of rare words

---

64. Word embeddings vs token embeddings

Older NLP discussions often say "word embedding."

Modern GPT systems are more accurately described as using:

token embeddings

because a token can be:

• a whole word

• part of a word

• punctuation

• whitespace-related representation

• a byte-derived unit, depending on tokenizer design

So:

```
token ID
→ embedding vector
```

is the modern mental model.

---

65. Full GPT mathematical intuition

Now connect everything.

Suppose the input tokens are:

```
[t1, t2, t3, t4]
```

Step 1: token embedding

Each token becomes a vector:

```
t1 → e1
t2 → e2
t3 → e3
t4 → e4
```

Step 2: position information

Each vector receives positional information.

Now:

```
x1, x2, x3, x4
```

contain token + position information.

Step 3: Query, Key, Value

Each vector produces:

```
Q
K
V
```

Step 4: compare

Each query compares against earlier keys.

Step 5: mask future tokens

Future positions are blocked.

Step 6: softmax

Scores become attention probabilities.

Step 7: retrieve information

Attention probabilities mix the Values.

Step 8: residual connection

Attention result is added to the original representation.

Step 9: feed-forward network

Each token representation is transformed independently.

Step 10: another residual connection

The FFN output is added back.

Step 11: repeat

Many Transformer blocks are stacked.

Step 12: vocabulary projection

Each final representation produces a score for every token.

Step 13: cross entropy

The model is penalized if the correct next token receives low probability.

---

66. One complete numerical toy example

Use only 3 tokens:

```
I
love
AI
```

Assume each token has a 2-dimensional vector.

```
I     = [1.0, 0.0]
love  = [0.0, 1.0]
AI    = [1.0, 1.0]
```

Suppose we process `AI`.

Let:

```
Query(AI) = [1, 1]
```

and keys:

```
Key(I)    = [1, 0]
Key(love) = [0, 1]
Key(AI)   = [1, 1]
```

Dot products:

```
AI query · I key
= 1×1 + 1×0
= 1

AI query · love key
= 1×0 + 1×1
= 1

AI query · AI key
= 1×1 + 1×1
= 2
```

So raw scores are:

```
[1, 1, 2]
```

Softmax gives the largest weight to the third position.

Then the model takes a weighted mixture of the Values.

The important idea is not the exact probability.

It is:

The representation of AI is built using information from the tokens it attends to.

---

67. Why attention is powerful compared with RNN memory

RNN:

```
past information
→ compressed into hidden state
→ passed forward
```

Attention:

```
current token
→ directly checks earlier token representations
```

So attention creates a shorter information path.

Instead of:

```
token 1
→ token 2
→ token 3
→ token 4
→ token 5
```

information can travel more directly:

```
token 5
→ token 1
```

This helps with long-range dependencies.

---

68. The Transformer does not "understand" in a human sense

It is important not to anthropomorphize the model.

A Transformer is a mathematical function.

It learns parameters that transform token representations so that next-token prediction becomes accurate.

At scale, these learned representations can encode:

• syntax

• semantics

• factual associations

• patterns

• code structure

• reasoning-like behaviors

But the underlying computation is still numerical.

---

69. Why larger models can become capable

A GPT model has parameters.

Parameters are learned numbers.

During training, the optimizer changes them so that the model becomes better at predicting the training distribution.

Scaling can provide:

• more parameters

• more training tokens

• more compute

• larger context

• better data

• improved architectures

• better optimization

The relationship is not simply:

"more parameters = intelligence"

A useful mental model is:

Capacity + data + optimization + architecture + compute + training quality work together.

---

70. Parameter count

For an embedding table:

```
parameters = vocabulary × embedding_dimension
```

For a linear layer:

```
parameters ≈ input_dimension × output_dimension
```

plus biases if used.

For a GPT model, parameter count comes mainly from:

• token embeddings

• attention projections

• feed-forward layers

• normalization parameters

• output projection

---

71. Rough Transformer block parameter intuition

Suppose:

```
model dimension = D
```

Q/K/V together approximately require:

```
3 × D × D
```

Output projection:

```
D × D
```

So attention projections are roughly:

$$4D^2$$

The feed-forward network often expands to about:

$$4D$$

and comes back to:

$$D$$

So its two large matrices together are roughly:

$$8D^2$$

Therefore a standard Transformer block has a large number of parameters proportional to:

$$D^2$$

This is why increasing hidden dimension becomes expensive.

---

72. Memory usage during training

Your GPU does not only store model parameters.

It also needs memory for:

- gradients
- optimizer state
- activations
- temporary tensors
- attention matrices

This is why training a model can require much more VRAM than simply storing its weights.

---

73. Why attention becomes expensive for long context

If sequence length is:

$$T$$

the attention score matrix has roughly:

$$T \times T$$

entries.

So doubling sequence length roughly quadruples the number of pairwise attention positions.

This is why long-context models require specialized techniques and optimized kernels.

---

74. Training vs inference

Training

The model sees many target positions simultaneously.

Causal masking prevents future leakage.

This allows highly parallel GPU computation.

Inference

Generation is autoregressive.

The model produces:

```
token 1
→ token 2
→ token 3
→ token 4
```

one new token at a time.

Modern implementations use techniques such as KV caching so they do not recompute everything from scratch for every generated token.

---

75. KV cache intuition

During generation, earlier Keys and Values usually do not need to be recomputed.

The model stores them.

When a new token arrives:

```
new Query
+
stored Keys
+
stored Values
```

are used to calculate the next output.

This can dramatically improve generation efficiency.

---

76. From your small GPT project to a real GPT

Your learning project can be understood as:

```
corpus
 ↓
tokenizer
 ↓
```

```
token IDs
 ↓
training batches
 ↓
token embeddings
 ↓
positional information
 ↓
causal self-attention
 ↓
multi-head attention
 ↓
feed-forward network
 ↓
Transformer blocks
 ↓
vocabulary logits
 ↓
cross entropy
 ↓
backpropagation
 ↓
optimizer
 ↓
updated parameters
```

Then:

```
prompt
 ↓
tokenizer
 ↓
token IDs
 ↓
GPT
 ↓
logits
 ↓
sampling
 ↓
new token
 ↓
append token
 ↓
repeat
```

That is the complete loop.

---

77. What you should implement yourself

For learning, implement these manually at least once:

Level 1

• tokenizer

• vocabulary

• embeddings

• softmax

• cross entropy

Level 2

• vanilla RNN

• RNN language model

• LSTM language model

Level 3

• single-head attention

• causal mask

• Q/K/V projections

Level 4

• multi-head attention

• feed-forward network

• residual connections

• LayerNorm

Level 5

• complete Transformer block

• GPT model

• training loop

• generation

Level 6

• BPE tokenizer

• checkpointing

• validation loss

• learning-rate scheduling

• mixed precision

• gradient accumulation

Level 7

• RoPE

• RMSNorm

• SwiGLU

• KV cache

• efficient attention

• distributed training concepts

---

78. Recommended learning order

Do not jump directly from ANN to a 7B model.

Use this sequence:

```
1. Embedding
2. Simple RNN
```

```
3. RNN language model
4. LSTM
5. Single-head attention
6. Causal attention
7. Multi-head attention
8. Transformer block
9. Tiny GPT
10. BPE tokenizer
11. Train on your own corpus
12. Add validation
13. Add generation controls
14. Study RoPE
15. Study RMSNorm
16. Study modern FFNs such as SwiGLU
17. Study KV cache
18. Study efficient attention
19. Study distributed training
20. Study modern LLM training
```

---

79. A practical project sequence

Project 1 — Word embeddings

Goal:

```
token ID → vector
```

Learn:

• `nn.Embedding`

• tensor shapes

• gradient updates

Project 2 — RNN text predictor

Dataset:

```
small text file
```

Build:

```
Embedding
→ RNN
→ Linear
→ CrossEntropyLoss
```

Goal:

Predict the next character/token.

Project 3 — LSTM

Replace:

```
RNN
```

with:

```
LSTM
```

Compare:

• training loss

• validation loss

• generated text

• sequence length

Project 4 — Attention from scratch

Do not use:

```
nn.MultiheadAttention
```

initially.

Implement:

```
Q
K
V
→ scores
→ scaling
→ causal mask
→ softmax
→ weighted values
```

This project will make Transformers much easier.

Project 5 — Transformer block

Implement:

```
LayerNorm
→ causal attention
→ residual
→ FFN
→ residual
```

Project 6 — Tiny GPT

Build:

```
Tokenizer
→ Embedding
→ Transformer
→ LM head
```

Train it on a small corpus.

---

80. How to read the mathematics without getting confused

When you see:

```
y = Wx + b
```

read:

"A matrix transforms the input, then a bias is added."

When you see:

```
softmax(z)
```

read:

"Turn scores into probabilities."

When you see:

```
QK■
```

read:

"Compare every query with every key."

When you see:

```
softmax(QK■ / √d)V
```

read:

"Compare → scale → convert to probabilities → retrieve information."

When you see:

```
x + f(x)
```

read:

"Keep the original information and add the newly processed information."

This translation habit is extremely useful.

---

81. The most important equations to understand

You do NOT need to memorize dozens of equations.

Understand these.

Embedding

```
token ID → learned vector
```

RNN

```
new memory =
tanh(
    current input effect
    +
    old memory effect
)
```

LSTM

```
new memory =
kept old memory
+
selected new information
```

Attention

```
attention =
softmax(
    query-key similarity
)
×
values
```

Transformer

```
representation
→ attention
→ residual
→ feed-forward
→ residual
```

GPT objective

```
make the correct next token highly probable
```

Loss

```
average negative log probability
of the correct next token
```

If these ideas are clear, the dense equations become much easier later.

---

82. Common beginner mistakes

Mistake 1

Trying to memorize equations.

Better

Understand what each equation does.

Mistake 2

Learning Transformers without understanding matrix multiplication.

Better

Practice tensor shapes.

Mistake 3

Thinking embeddings are manually assigned meanings.

Better

Embeddings are learned parameters.

Mistake 4

Thinking attention is simply "memory."

Better

Attention dynamically retrieves information from other token representations.

Mistake 5

Forgetting causal masking.

Without masking, a GPT training example could accidentally see future tokens.

Mistake 6

Applying softmax before PyTorch CrossEntropyLoss.

Usually:

```
loss = F.cross_entropy(logits, targets)
```

not:

```
loss = F.cross_entropy(
    torch.softmax(logits, dim=-1),
    targets
)
```

Mistake 7

Changing many hyperparameters simultaneously.

Then you cannot tell what caused improvement or degradation.

---

83. How backpropagation flows through GPT

The final loss depends on the predicted logits.

The chain is:

```
loss
 ↓
vocabulary projection
 ↓
Transformer block N
 ↓
Transformer block N-1
 ↓
...
 ↓
attention
 ↓
Q/K/V projections
 ↓
embeddings
 ↓
token IDs' embedding rows
```

The optimizer then changes parameters to reduce future loss.

This includes:

• embedding vectors

• Q/K/V matrices

• FFN matrices

• normalization parameters

• output projection

---

84. A useful mental model of training

Imagine GPT makes this prediction:

```
Correct token: "cat"

Model probability:
cat = 0.02
```

Loss is high.

Backpropagation asks:

Which parameters contributed to giving "cat" such a low score?

The gradient travels backward.

The optimizer adjusts those parameters.

After many examples, the model learns parameter configurations that produce better predictions.

That is training.

---

85. What the Transformer layers learn

There is no guarantee that every layer has a single human-readable job.

But learned representations can often exhibit patterns related to:

• local syntax

• long-distance dependencies

• semantic relationships

• entity information

• phrase structure

• task-relevant features

The model is not programmed with rules such as:

```
if pronoun == "it":
    find nearest noun
```

Instead, such behavior can emerge from learned numerical transformations.

---

86. Why multiple layers?

One attention layer can combine information.

Another layer can transform that information.

Another layer can combine the transformed information again.

So a stack creates increasingly complex representations.

Conceptually:

```
raw token
```

```
 ↓
local/contextual representation
 ↓
richer contextual representation
 ↓
semantic/syntactic representation
 ↓
prediction-oriented representation
```

Do not assume these stages are perfectly separated in real models. They overlap.

---

87. Why multiple heads?

Suppose a sentence contains several relationships.

One head might strongly attend to nearby words.

Another might attend to a subject.

Another might attend to a repeated entity.

Another might learn punctuation or structural patterns.

These interpretations are useful intuitions, but individual heads do not necessarily have one fixed human-readable purpose.

---

88. Encoder vs decoder

Original Transformer architecture contains:

```
Encoder
+
Decoder
```

Encoder

Good for representing an entire input sequence.

Examples historically include translation encoders and BERT-style models.

Decoder

Generates output autoregressively.

GPT uses the decoder side conceptually, specifically a decoder-only Transformer.

---

89. GPT vs BERT

GPT

```
decoder-only
causal attention
```

```
next-token prediction
generative
```

BERT

```
encoder-only
bidirectional context
masked-token style pretraining
representation-focused
```

This distinction is foundational in modern NLP.

---

90. Why GPT can generate text

The model only needs one basic capability:

```
P(next token | previous tokens)
```

Generation repeatedly applies that capability.

Suppose:

```
P(cat | The) = 0.3
P(dog | The) = 0.2
...
```

Choose or sample a token.

Then:

```
P(is | The cat)
```

Then:

```
P(sleeping | The cat is)
```

The same model is used again and again.

---

91. A deeper probability view

A complete sentence can be thought of as:

```
P(token1, token2, token3, ...)
```

The model decomposes this into conditional predictions:

```
P(token1)
×
P(token2 | token1)
×
P(token3 | token1, token2)
×
...
```

GPT learns to model these conditional probabilities.

This is the probabilistic foundation of autoregressive language modeling.

---

92. Why the model can answer questions without a special "question module"

If the training data contains many patterns like:

```
Question → Answer
```

the model can learn statistical structures that map the context toward likely continuations.

Instruction tuning and later training stages can make these behaviors much more useful and aligned.

But the core GPT mechanism remains next-token prediction.

---

93. Pretraining vs instruction tuning

Modern LLM systems often have multiple training stages.

Pretraining

Large amounts of text.

Main objective:

```
predict next token
```

Instruction tuning

Examples of:

```
user instruction
→ desired response
```

The model learns to follow instructions more effectively.

Preference/alignment stages

Additional methods can optimize behavior according to human or model-generated preferences and safety objectives.

These stages are conceptually separate from the basic Transformer architecture.

---

94. What makes a modern GPT different from your TinyGPT?

Your TinyGPT might have:

```
6 layers
256 dimensions
```

```
8 heads
small dataset
small tokenizer
```

A production LLM can additionally use:

• vastly more parameters

• vastly more training data

• sophisticated tokenization

• modern positional methods

• modern normalization

• gated feed-forward networks

• optimized attention

• distributed training

• mixed precision

• sophisticated data pipelines

• checkpointing

• parallelism

• inference optimization

• instruction tuning

• evaluation systems

• safety systems

The core conceptual pipeline remains recognizable.

---

95. A modern Transformer block

A modern decoder block may conceptually look like:

```
input
 ↓
normalization
 ↓
positional-aware Q/K processing
 ↓
causal attention
 ↓
residual addition
 ↓
normalization
 ↓
gated feed-forward network
 ↓
residual addition
 ↓
output
```

Specific modern architectures vary.

Do not treat one implementation as the definition of all GPT models.

---

96. RoPE, RMSNorm, and SwiGLU

These are useful next topics after your basic GPT.

RoPE

Provides position information through rotations applied to Q/K components.

RMSNorm

A normalization method related to LayerNorm but without the same mean-centering operation.

SwiGLU

A gated feed-forward architecture that can improve model capacity/efficiency in many modern Transformer designs.

Study these **after** basic attention and Transformer blocks.

---

97. Your mathematical learning roadmap

Do not start with advanced papers.

Use:

Stage A

Vectors and matrices

Learn:

```
matrix × vector
matrix × matrix
transpose
dot product
dimensions
```

Stage B

Neural-network calculus

Learn:

```
chain rule
gradient
Jacobian intuition
```

Stage C

Probability

Learn:

```
probability distribution
conditional probability
```

```
log probability
entropy
cross entropy
```

Stage D

Sequence models

Learn:

```
RNN recurrence
BPTT intuition
LSTM gates
```

Stage E

Attention

Learn:

```
dot product
Q/K/V
softmax
masking
scaled attention
```

Stage F

Transformers

Learn:

```
multi-head attention
residuals
normalization
FFN
positional information
```

Stage G

LLMs

Learn:

```
tokenization
GPT objective
generation
sampling
KV cache
scaling
training systems
```

---

98. A 12-week practical plan

Week 1

Review:

• vectors
• matrices
• matrix multiplication
• softmax
• cross entropy

Build:

• embedding demo

Week 2

Learn:

• sequence prediction
• RNN

Build:

• character-level RNN

Week 3

Learn:

• BPTT
• vanishing gradients

Build:

• RNN language model

Week 4

Learn:

• LSTM gates

Build:

• LSTM language model

Week 5

Learn:

• Query

• Key

• Value

• dot-product attention

Build:

• single-head attention from scratch

Week 6

Learn:

• causal mask

• multi-head attention

Build:

• causal multi-head attention

Week 7

Learn:

• residuals

• LayerNorm

• FFN

Build:

• Transformer block

Week 8

Build:

• TinyGPT

Week 9

Add:

• BPE tokenizer

• better dataset pipeline

• train/validation split

Week 10

Add:

• checkpointing

• generation

• temperature

• top-k

• top-p

Week 11

Study:

• RoPE

• RMSNorm

• SwiGLU

• KV cache

Week 12

Study:

• LLM scaling

• mixed precision

• distributed training

• inference optimization

• evaluation

---

99. Exercises

Exercise 1

Create an embedding:

```
nn.Embedding(100, 16)
```

Feed 5 token IDs.

Find the output shape.

Exercise 2

Build a character-level RNN.

Dataset:

```
hello world
```

Predict the next character.

Exercise 3

Replace RNN with LSTM.

Compare losses.

Exercise 4

Implement attention without:

```
nn.MultiheadAttention
```

Use only:

```
nn.Linear
torch.matmul
torch.softmax
```

Exercise 5

Print the causal attention matrix.

Verify that future positions are zero after masking and softmax.

Exercise 6

Build one Transformer block.

Verify every tensor shape.

Exercise 7

Build TinyGPT.

Train it on a small text file.

Exercise 8

Overfit a tiny dataset.

This is important.

If your model cannot memorize a tiny dataset, something is probably wrong in the implementation.

---

100. Debugging checklist

When GPT training fails, check:

Data

```
Are token IDs within vocabulary range?
```

Shapes

```
Does embedding output have [B, T, C]?
```

Targets

```
Is target exactly one token shifted relative to input?
```

Mask

```
Can a token see future positions?
```

Loss

```
Are raw logits passed to CrossEntropyLoss?
```

Device

```
Are model and tensors on the same device?
```

Learning rate

Too high:

```
loss becomes NaN
```

Too low:

```
loss barely moves
```

Data quality

If the corpus is tiny or repetitive, the model may simply memorize it.

---

101. The most important debugging tool: shapes

Print:

```
print("tokens:", tokens.shape)
print("embedding:", x.shape)
print("q:", q.shape)
print("k:", k.shape)
print("v:", v.shape)
print("scores:", scores.shape)
print("attention:", weights.shape)
print("logits:", logits.shape)
print("targets:", targets.shape)
```

Many Transformer bugs are shape bugs.

---

102. A mental model for every tensor

When you see:

```
[B, T, C]
```

think:

```
B = how many examples
T = how many tokens
C = how many numbers represent each token
```

When you see:

```
[B, H, T, T]
```

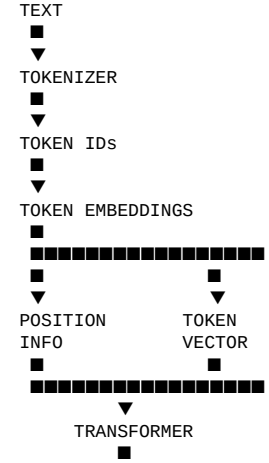
think:

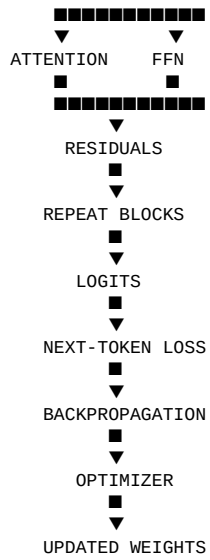
```
B = examples
H = attention heads
T = query positions
T = key positions
```

This habit makes attention much less intimidating.

---

103. The complete conceptual map





Generation:

```
PROMPT
 ↓
TOKENIZE
 ↓
GPT
 ↓
NEXT TOKEN PROBABILITIES
 ↓
SAMPLE
 ↓
APPEND TOKEN
 ↓
GPT AGAIN
 ↓
REPEAT
```

---

104. Final understanding test

Before saying:

"I understand Transformers"

you should be able to answer these questions without looking them up:

1. Why do we need embeddings?
2. Why does an RNN need hidden memory?
3. Why can RNNs struggle with long dependencies?
4. What problem does LSTM try to solve?
5. What does the forget gate do?
6. What does attention do?
7. What are Query, Key, and Value?
8. Why do we divide attention scores by the square root of the head dimension?
9. Why is softmax used?
10. Why does GPT need a causal mask?
11. Why does multi-head attention use multiple heads?
12. Why do Transformers use residual connections?

13. What does the feed-forward network do?

14. Why is positional information needed?

15. Why is GPT called decoder-only?

16. What is next-token prediction?

17. Why is the target sequence shifted by one token?

18. What are logits?

19. Why can CrossEntropyLoss receive raw logits?

20. Why is generation sequential even though Transformer training is highly parallel?

21. What is a KV cache?

22. Why does long context increase attention cost?

23. What is the difference between pretraining and instruction tuning?

If you can explain these in your own words, you have the conceptual foundation for studying modern LLMs.

---

105. Final takeaway

The entire story can be compressed into one progression:

RNN

"I read tokens one by one and carry a memory."

LSTM

"I have gates that help me keep or discard information."

Attention

"Instead of relying only on one carried memory, I can directly look at relevant token representations."

Transformer

"I can repeatedly use attention and feed-forward transformations without processing the whole sequence through a recurrent chain."

GPT

"I use a decoder-style Transformer with causal attention to predict the next token."

Modern GPT

"Scale this architecture with better tokenization, positional methods, normalization, optimization, data, training infrastructure, and inference techniques."

That is the conceptual bridge from basic ANN knowledge to modern language models.

---

106. One-page cheat sheet

```
EMBEDDING
token ID → learned vector

RNN
current input + old memory → new memory

LSTM
old memory
   ↓
forget gate
   +
new candidate
   ↓
new memory
   ↓
output gate
   ↓
hidden state

ATTENTION
Q = what am I looking for?
K = what information is available?
V = what information should I retrieve?

attention:
compare Q with K
→ scale
→ softmax
→ weighted sum of V

MULTI-HEAD
many attention mechanisms
→ concatenate
→ output projection

TRANSFORMER BLOCK
normalize
→ attention
→ residual
→ normalize
→ FFN
→ residual

GPT
tokens
→ embeddings
→ positional information
→ causal Transformer blocks
→ logits
→ next-token probabilities

TRAINING
input:
The cat is

target:
cat is sleeping

LOSS
penalize low probability for correct next token

GENERATION
predict
→ append
→ predict
→ append
→ repeat
```

---

107. Suggested next practical milestone

After reading this document, do not immediately move to a large LLM.

The best next exercise is:

Build a tiny GPT from scratch using only PyTorch, train it on your own text file, and inspect every tensor shape and every attention matrix.

The objective is not to make a useful chatbot.

The objective is to make every step from:

```
text
→ token
→ embedding
→ Q/K/V
→ attention
→ Transformer
→ logits
→ loss
→ gradient
→ updated weight
```

feel mechanically understandable.

Once that pipeline becomes intuitive, modern LLM papers become dramatically easier to read.